NetResQ — Team Presentation & Defense Plan

Structured 4-Member Project Walkthrough, Speaking Scripts & Viva Q&A

Member	Primary Topic	Key Code Files	Live Dashboard Action
Member 1	Problem Statement, SDN Philosophy, OSI Layer 3	`backend/app/config.py`	Overview cards, Header status, Triangle intro
Member 2	Metric Normalization, Composite Cost, Dijkstra Proof	`app/algorithms/quality_aware.py`	Routing Analysis tab, Hop breakdown table
Member 3	Self-Healing Engine, Failover Timing, REST API	`app/services/simulation_service.py`	Fail Link button, 0.48ms recalc badge, Event log
Member 4	Operations UI, Demo Congestion, Ryu/Mininet	`frontend/src/components/TopologyView.tsx`	Demo Congestion button, Weight sliders, Reset

Member 1: Project Lead & Architecture Specialist

BACKGROUND & ARCHITECTURE (~2.5 MINS)

CORE CONCEPTS TO EXPLAIN:

- **The "Blind Shortest Path" Flaw:** Traditional OSPF/RIP routing follows static hop counts. If a link has 65ms delay and 20% loss, standard Dijkstra still sends packets through it because the configured cost is static.
- **The NetResQ Solution:** Adaptive self-healing routing at **OSI Layer 3** that continuously evaluates link quality.
- **SDN Architecture:** Decoupling the **Control Plane** (decision logic in FastAPI + NetworkX) from the **Data Plane** (OpenFlow switches forwarding packets).
- **Network Graph:** 3-switch triangle core (S1, S2, S3) with 4 attached hosts (H1, H2 on S1; H3 on S2; H4 on S3).

LIVE DASHBOARD ACTIONS:

- Open `http://localhost:5173`. Point to header: "NetResQ", "BACKEND LIVE", "SIMULATION MODE".
- Highlight the Overview metric cards: Switches Online (3), Healthy Links (7), Failures (0), Health Score (100%).

SPEAKING SCRIPT / PITCH:

> *"Good morning, respected evaluators. I am [Name], presenting the architecture of NetResQ. In traditional networks, routing protocols like OSPF calculate paths using static administrative hop costs. When an intermediate link experiences congestion or packet drops without a physical cable cut, standard routing is completely blind to it. NetResQ solves this by introducing dynamic telemetry and self-healing at OSI Layer 3. We use an SDN architecture where our FastAPI control plane maintains an authoritative NetworkX graph model. I will now hand over to [Member 2] to explain the mathematical routing algorithm."*

KEY VIVA QUESTIONS & ANSWERS:

Q: Which OSI layer does NetResQ operate at?
A: OSI Layer 3 (Network Layer), responsible for end-to-end path determination across switches and subnets.

Q: What is the separation of planes in SDN?
A: Decoupling the decision-making logic (Control Plane) from underlying packet forwarding hardware (Data Plane).

Member 2: Routing Algorithms & Math Lead

ALGORITHMS & MATH (~3 MINS)

CORE CONCEPTS TO EXPLAIN:

- **Why Normalization is Required:** Milliseconds (e.g. 60ms), percentages (e.g. 18%), and administrative costs cannot be summed directly. NetResQ normalizes each metric into dimensionless [0, 1] scales.
- **Normalization Formulas:** `norm_lat = lat / 50ms`, `norm_loss = loss% / 100`, `norm_util = util% / 100`.
- **Composite Cost Formula:** `link_cost = (w_cost × C) + (w_lat × norm_lat) + (w_loss × norm_loss) + (w_util × norm_util)`.
- **Proof of Dijkstra Correctness:** Since all weights ≥ 0 and metrics ≥ 0, and cost floor is 0.001, no negative cycles can exist. Dijkstra is mathematically guaranteed to find the global minimum cost.
- **Worked Example (H1 → H4):** Path A (Direct S1-S3, 18% loss): Cost = **6.12**.
  Path B (Detour via S2, 0% loss): Cost = **4.79**.
  Quality-Aware picks Path B because 4.79 < 6.12, saving 54ms delay and 18% loss!

LIVE DASHBOARD ACTIONS:

- Click on the **"Routing Analysis & Proof"** tab.
- Walk through side-by-side comparison cards and point to the per-link cost breakdown table.

SPEAKING SCRIPT / PITCH:

> *"I am [Name], and I will explain the mathematical core of NetResQ. To overcome static hop counting, we developed a multi-metric composite cost function. Because milliseconds and percentages have incompatible units, we first normalize latency against a 50ms reference, and loss and utilization into decimals between 0 and 1. We weight packet loss higher at 2.0 to aggressively penalize drop-prone links. Crucially, because all edge costs are strictly positive (≥ 0.001), Dijkstra's algorithm is mathematically guaranteed to find the global optimum. As seen on our screen, when direct link S1-S3 degrades, our formula proves that a 4-hop clean path has a lower composite cost (4.79) than a degraded 3-hop path (6.12). I will now pass to [Member 3] to show how the self-healing failover engine operates."*

KEY VIVA QUESTIONS & ANSWERS:

Q: Why not use Bellman-Ford instead of Dijkstra?
A: Bellman-Ford is slower (O(V × E)) and only needed for negative edge weights. Because our composite cost function guarantees strictly positive costs, Dijkstra (O(E + V log V)) is faster and globally optimal.

Member 3: Self-Healing Engine & API Specialist

FAILOVER & BACKEND (~3 MINS)

CORE CONCEPTS TO EXPLAIN:

- **Failure Handling:** Failed links are marked `DOWN` in the authoritative simulation adapter. Graph building skips them: `if link.status == 'down': continue`.
- **Automated Failover:** Dijkstra recalculates active routes in **sub-millisecond time (~0.48ms)**.
- **Monotonic Clock Benchmarking:** Measures recalculation time via Python's `time.perf_counter()`.
- **Network Partition Handling:** If all paths fail, `networkx.has_path()` returns False; API cleanly returns status `"no_path"` without crashing.
- **FastAPI Endpoints:** `GET /api/topology`, `POST /api/links/{id}/fail`, `POST /api/links/{id}/restore`, `GET /api/events`.

LIVE DASHBOARD ACTIONS:

- Go to the **"Network Topology & Control"** tab.
- Click core link `L_S1_S3`, click red **"Fail Link"** button. Show the link turning red-dashed and route switching to S2 in **0.48ms**!
- Open the **"Operational Events"** tab to show the recorded warning event. Click **"Restore Link"** to show recovery.

SPEAKING SCRIPT / PITCH:

> *"I am [Name], in charge of the self-healing failover engine and backend services. Our backend is built on FastAPI and Uvicorn. When a link fails, our SimulationService immediately excludes that edge from the NetworkX graph. Notice on our screen: as I click 'Fail Link' on L_S1_S3, the topology instantly turns red-dashed, and our self-healing engine detects the loss, recalculates the path through S2, and restores connectivity in just 0.48 milliseconds. We measure this software execution time using Python's monotonic clock. I now hand over to [Member 4] to demonstrate congestion scenarios and our SDN emulation."*

KEY VIVA QUESTIONS & ANSWERS:

Q: What is the difference between software rerouting time and physical recovery time?
A: Software route computation (~0.5ms) is CPU time to prune the graph and run Dijkstra. Physical recovery in hardware includes carrier loss detection and OpenFlow OFPFlowMod rule installation (typically 20ms to 100ms).

Member 4: UI/UX, Live Demo & SDN Emulation Lead

DEMO & SDN INTEGRATION (~3 MINS)

CORE CONCEPTS TO EXPLAIN:

- **Frontend Architecture:** React 19 + TypeScript + Vite. Custom responsive SVG canvas rendering for nodes and edges with zero external chart bloat.
- **Animated Active Route:** Cyan glowing packet flow animation (`active-route-line`).
- **Congestion Injection Demo:** Injects 65ms latency and 18% loss on `L_S1_S3`. Shows Quality-Aware routing dynamically detouring via S2 while Baseline Dijkstra stays trapped on congested link.
- **Dynamic Weight Sliders:** Settings tab allows tuning loss and latency weights live.
- **Real SDN Integration (Ryu + Mininet):** Scripts provided for Linux/WSL2 (`mininet_topology.py` and `ryu_controller.py`).

LIVE DASHBOARD ACTIONS:

- Click the amber **"Demo Congestion"** button in the header. Show active route detouring around congested link.
- Navigate to **"Settings & Weights"** tab and drag the Packet Loss Weight slider.
- Click **"Reset Scenario"** to restore clean baseline.

SPEAKING SCRIPT / PITCH:

> *"I am [Name], and I lead the dashboard engineering and SDN integration. Our UI is built in React 19 and TypeScript, designed to resemble a carrier-grade Network Operations Center. The topology you see is rendered as an interactive vector SVG, where the active route is highlighted with animated flowing packets. Now, let me demonstrate our primary test scenario: I will click 'Demo Congestion'. This injects 65ms latency and 18% packet loss on direct link L_S1_S3. Watch the active route: baseline Dijkstra remains trapped on the congested link because it only counts hops, but NetResQ dynamically switches to the 4-hop route via S2 to safeguard throughput and latency. Additionally, NetResQ includes Mininet and Ryu controller integration scripts for Ubuntu and WSL2 environments. This concludes our presentation, and we are ready for your questions."*

KEY VIVA QUESTIONS & ANSWERS:

Q: How does LLDP discovery work in an SDN controller like Ryu?
A: The controller sends LLDP packet-out messages to all switch ports. When neighbor switches forward the packet-in back to the controller, the controller maps the adjacency between switch datapath IDs and port numbers.